

ARM-based Palm OS Overview

David Fedor, Architect/Evangelist



Palm OS® V. 5 Detailed Overview



The Big Picture: Palm OS® 5

- Fully ARM-native, 32-bit OS
- New hardware
- Well-behaved applications “just work”
- Not quite the same as “5.0” at PalmSource 2000
  - Modularized for faster delivery, stability
  - Smoother transition, and earlier schedule
  - Palm OS 5 delivers power of ARM, and modular delivery of new APIs
  - New tools and modules coming throughout 2002



Why ARM?

- Speed (large range available)
- Power (good MIPS-to-Watts ratio)
- Cost
- Capability, integration with licensee components
- Availability (from many suppliers)
- Standard chip in several markets, e.g., phones

What, When, Where…

- GM OS software to licensees in first half of 2002
- Devices to follow
  - We can't disclose the plans of our licensees
  - Sometimes very soon after OS GM, sometimes they need more time for their hardware and/or integration
- OS supports a wide range of products
  - Balance price, battery life, memory and CPU speed
  - Permutations of add-on and/or integrated hardware and software



Topics for today

- Palm OS on ARM
  - The OS, the hardware, the emulation
- Making sure your apps work
  - Testing tools, compatibility, PalmOSGlue…
  - Details on the Pace environment
  - Working well with new OS features (passively)
- New capabilities and APIs
  - Taking advantage of new OS features (actively)

The Fundamentals

- ARM chip compared to 68k Dragonball
  - Instruction set, endianness of numbers…
- We have a 68k compatibility environment so existing, well-behaved applications just work
- Everything is in native ARM except the app itself,* therefore high performance

  * the app might have native ARM code too; see talk 112.
- Common bugs and hacking can cause emulation trouble
  - Clean up your apps tonight!
- User experience is great: totally natural, full speed

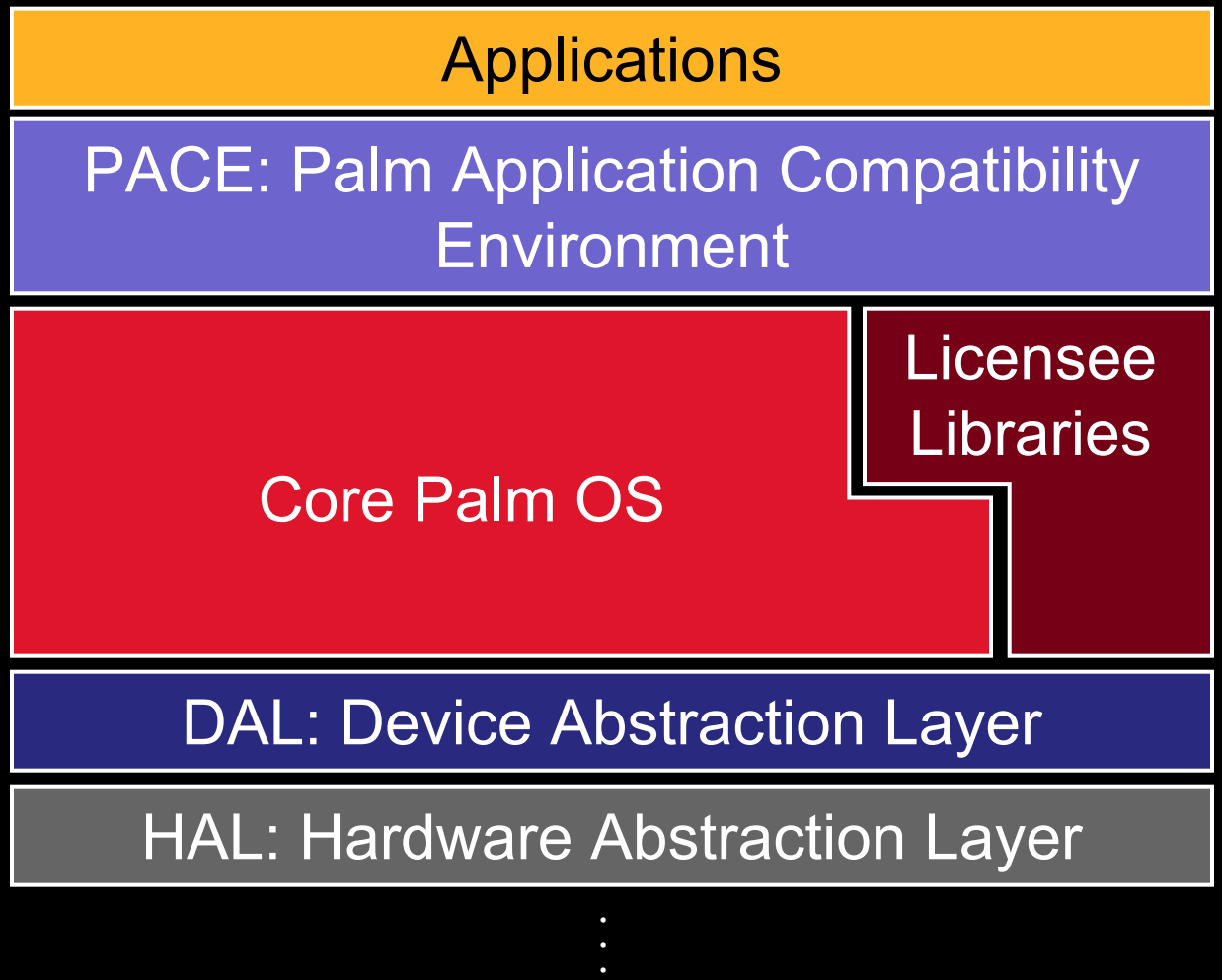
Hardware

- Support for ARM's 4T architecture
  - There are lots in this category, many vendors
  - CPU speeds anywhere from 18 MHz to 1GHz today
  - OS might be Thumb or not; uses interworking
- Palm OS Ready program
  - Intel, Motorola, TI will provide chips+software
- Internally using several boards, both custom and "off-the-shelf"

Compatibility With 68k Devices and Data

- Applications have 68k code, and read/write their data and prefs in Big-Endian mode
  - Same as always
- All saved data, and all objects in memory seen by applications are Big-Endian
- 68k Emulator (“PACE”) translates parameters, objects, etc. into little-endian when calling an OS routine
- Therefore, applications do not have to change (yet) to handle new endianness

Palm OS 5 Block Diagram





Palm OS Simulator

- Implementation of Palm OS Platform code as a normal Windows application
  - (System.dll, UI.dll…)
  - Applications are 68k PRCs in the ".ROM" file
- Implemented on top of abstraction layers; the OS doesn't know or care what it is running on
- Windows is little-endian, like the ARM chips

Palm OS Simulator (Cont.)

- Includes PACE and can talk to debuggers
- Different from Palm OS Emulator
  - Not hardware emulation
  - Fewer checks for app bugs
  - Some great, unique debugging features
- Won't catch processor-specific bugs
  - Unaligned reads/writes
  - Memory locations are different



Palm OS 5 Simulator Demo



Tools

- Use all your existing tools to build PRCs
  - CodeWarrior
  - gcc with Prc-tools
  - Etc.
- Palm OS Emulator with 4.x debug ROMs
- Palm OS Simulator
- PalmOSGlue
- Debugging on a board or device
- (New toolchain coming)



Handling Performance Problems

- Most applications are plenty fast on ARM
- Some portions of some can be slower
  - Depends on many factors
  - Worst case is when an app has a core algorithm that does all the work inside itself
  - Many things that were impractical on a Dragonball are now possible

Native ARM Coding

- These portions can be moved to be native ARM
  - Not the full application, not very good for doing UI
  - Tools are preliminary, gcc-based, but functional
- Easy to make it conditional so the same PRC will work on 68k devices as well as on ARM-based devices
  - App checks `sysFtrNumProcessorID`
  - Calls either 68k, ARM or Simulator version of code

Hacking and Patching

- Since the OS is native ARM, little-endian and different under the hood, the existing SysSetTrapAddress() is unimplementable
- Lots of trap patching was done to add cool features
  - Often more just as a notification, not a replacement or modification of functionality
- New notifications provide equivalent power for most commonly hacked situations
- Sustainable patching mechanism coming

High-density Window Manager

- Natural and simple support for higher density screens
- Existing applications get benefits with no work
  - Better fonts, smooth buttons…
- Tools support multiple densities in a bitmap family, so OS picks “best” one for device
- API adopted by Sony; compatibility layer coming
- Details in talk 112

Sound

- Play standard Wave files
  - (Usually resources)
  - Uncompressed, or IMA ADPCM
- Play streamed raw samples
- Playback 16 streams at a time, 8 or 16 bit
- Recording of 1 or 2 channels, 8 or 16 bit
- Support for arbitrary sampling rates

- Details in talk 112!



Get Compatible With Palm OS 5

- “I really didn’t expect it to be that easy.”
  - Scott@red-mercury.com
- Need to fix apps so they work
- Might need to adjust performance
- Might need to fail gracefully
- More exciting: create new apps which used to be impossible!
- Details in talk 103 (next)



Doing Things Better

- It is time to move to the supported APIs
  - Use Sleep/wake notifications; don't hack SysSleep()
  - Use Glue instead of direct access to structures
  - Stop calling deprecated APIs
  - Don't access hardware without checking first
- Don't re-implement a system API to "optimize" it
  - Your own super-fast bitmap copying routine will now probably be slower than the system's!



To Be Clear:

Unsupported things that we said would no longer work in the “next OS release”:

- Access to internal structures and UI objects
- Access to the PIM records
- Patching traps
- Calling unsupported functions
- Hardware access, screen buffer access

To Be Clear, (Cont.):

Actual reality: which unsupported things will no longer work like they did before:

- NO: Access to internal structures and UI objects
  - (But you have new glue routines)
- OK: Access to the PIM records (for now)
- NO: Patching traps
  - (But notifications exist)
- NO: Calling unsupported functions
- NO: Hardware access, screen buffer access
  - Accessing an offscreen buffer is OK, and you have APIs to create them however you want



The Next Major OS Release

- Palm OS 5 didn't touch the existing APIs and programming model
  - The next major release will
- Support for native applications & libraries
  - Have to handle endianness issues, new compilers
  - Some API changes, many additions
  - Importance of wire formats

The Next Major OS Release (Cont.)

- Big new OS features, including a new runtime model
  - Event delivery, threading, memory protection…
- New sync and desktop capabilities
- PIM record formats; Data Mgr upgrades
- PACE is and will still be a great option for most applications



PalmSource®

Conference and Expo